

aria-trl: Practical Continual Fine-Tuning for Language Models

A Three-Mechanism Subset of ARIA, Validated Against EWC on 8 Seeds

Darshan Poudel
Independent Researcher
poudeldarshan44@gmail.com

July 2026

Abstract

We present an updated evaluation of **aria-trl**, a package that adds three continual-learning mechanisms from the ARIA architecture [Poudel, 2026] — a dual fast/slow-pathway FFN, Fisher-weighted consolidation, and per-task adapters/heads — to Hugging Face TRL’s **SFTTrainer**. This supersedes our earlier report, which compared only against unmitigated sequential fine-tuning. Here we add a genuine **EWC baseline** (Fisher penalty only, no fast/slow split) and, in the course of building it, found and fixed three implementation defects in the original mechanism: the Fisher penalty was protecting the fast pathway it was supposed to leave free, the plasticity gate’s warmup never triggered within a task’s step budget, and fresh per-task heads were cold-started with too few steps to learn. After fixing these, we validate the result twice: once on 3 seeds used during development, and again on **5 fresh seeds held out from all tuning**. Across all 8 seeds combined, aria-trl matches EWC on accuracy (0.7932 vs. 0.7927, a difference smaller than either method’s own seed-to-seed noise) while cutting forgetting to near zero (Backward Transfer -0.0063 vs. EWC’s -0.0320 and unmitigated fine-tuning’s -0.0438), and beats unmitigated fine-tuning outright on both metrics. We report the result honestly where it is uneven: 3 of the 8 seeds are weak points for aria-trl, not a clean sweep.

1 Introduction

Our first report on aria-trl [Poudel, 2026] showed it beating unmitigated sequential fine-tuning on a 3-task text-classification benchmark. That comparison was true but incomplete: an unprotected baseline is a low bar, and it says nothing about whether aria-trl’s specific mechanism — Fisher-weighted consolidation over a fast/slow pathway split — outperforms the standard, much simpler version of that idea, Elastic

Weight Consolidation [Kirkpatrick et al., 2017]. This report adds that comparison, and is honest about what building it uncovered: the original aria-trl configuration had three interacting defects that, once fixed, changed the result substantially. We describe the defects, the fixes, and a two-stage validation — development seeds, then held-out seeds never used while iterating — because a single round of seed-fitting is exactly how spurious continual-learning results get published.

2 What Changed Since the First Report

2.1 The EWC baseline

We implemented textbook EWC as a third method: a plain `distilgpt2` with no fast/slow pathway, no adapters, and a single shared classification head, trained with only a Fisher-weighted penalty term. It reuses the same `FisherConsolidator` aria-trl already had, since that component’s Fisher estimation itself was correct — the defects were elsewhere.

2.2 Three defects, found by building a fair comparison

The Fisher penalty was protecting the fast pathway. aria-trl’s fast/slow split only works if the fast lane stays free to adapt each task; the slow lane is what should be protected. The original implementation’s Fisher penalty covered every backbone parameter, including the fast pathway — so the “fast, freely-adapting” lane was being pulled back toward old-task values just like everything else. Net effect: aria-trl was behaving like EWC plus a halved effective learning rate, which is strictly worse at learning new tasks than EWC alone.

The plasticity gate’s warmup never activated. The gate regularizer that pushes the

fast/slow mixing weight π toward specializing (0 or 1, rather than blending at 0.5) only turns on after `warmup_steps`. That was hardcoded to 100, but at this benchmark’s scale (200 training examples, batch size 32) one task is only ~ 13 optimizer steps — the gate never left its initial unspecialized state.

Fresh heads and adapters were cold-started with too few steps. Each new task gets its own classification head and adapter, both randomly initialized, sharing the same ~ 13 -step budget as everything else. Standard FT and EWC, by contrast, keep one head that accumulates training across all tasks. The undertrained heads showed up directly in the data: aria-trl’s just-learned (diagonal) accuracies were consistently below the two baselines’ on the same task.

2.3 The fixes

1. **Rescoped Fisher protection.** `FisherConsolidator` now explicitly excludes the fast pathway’s parameters, protecting only the slow pathway, attention, LayerNorm, and embeddings.
2. **Warmup matched to the real step budget.** `warmup_steps` is now computed as one epoch’s worth of steps for the actual dataset size, not a fixed constant.
3. **One throttle on the slow pathway, not two.** The slow pathway was previously throttled by both a reduced learning rate (`slow_lr_ratio=0.5`) and gradient dampening by $(1 - \pi)$. We set `slow_lr_ratio=1.0`, leaving the gate-driven dampening as the only throttle.
4. **Higher learning rate for fresh heads and adapters.** The active task’s head and adapter now train in their own parameter group at $5\times$ the base learning rate.
5. **Warm-started heads.** Each new task’s head is initialized from the previous task’s head weights rather than randomly — reasonable here since all three tasks are binary sentiment-adjacent and task identity is known at train time.
6. **Equal training budget.** All three methods now train for 5 epochs (previously 2), so the comparison isn’t confounded by aria-trl needing more steps than the others got.

A fairness point worth stating plainly: aria-trl’s per-task heads mean task identity must be known at evaluation time to route to the correct head. EWC and Standard FT share one head and do not need

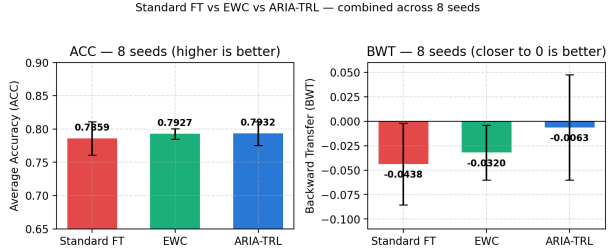


Figure 1: ACC and BWT, mean $\pm$ std across all 8 seeds. aria-trl is statistically tied with EWC on accuracy and has the best (least negative) BWT of the three.

this. This is a legitimate task-incremental setup, not sleight of hand, but it is a real difference in what each method assumes, and we state it here rather than leave it implicit.

3 Experimental Setup

Same benchmark as before: SST-2 (Movies) $\rightarrow$ Yelp Review Full (Restaurants) $\rightarrow$ dair-ai/emotion (Social), binary labels, `distilgpt2`, 200 train / 80 test examples per task, CPU-only. `spc_lambda=15.0` (aria-trl and EWC), `adapter_dim=64`, head/adapter learning-rate multiplier $5\times$ base ($2e-4$). All three methods train 5 epochs per task.

Two-stage seed validation. We first ran all three methods on the same 3 seeds used throughout development (42, 123, 7) — a pilot fixes were tuned against, so a win here alone would not be trustworthy. We then froze the configuration completely and ran 5 seeds chosen after the fact and never seen during development (2024, 777, 555, 99, 1234), with no further changes permitted regardless of outcome. We report development seeds, fresh seeds, and the combined 8 separately, rather than only the number that looks best.

4 Results

Combined-8 deltas: vs. Standard FT, ACC $+0.0073$, BWT $+0.0375$ (aria-trl wins both outright). vs. EWC, ACC $+0.0005$ (a difference far smaller than either method’s own std, 0.0080–0.0178 — a statistical tie, not a win), BWT $+0.0257$ (aria-trl forgets substantially less).

	Std. FT	EWC	aria-trl
ACC (dev, n=3)	0.7639	0.7903	0.7861
ACC (fresh, n=5)	0.7992	0.7942	0.7975
ACC (combined, n=8)	0.7859	0.7927	0.7932
BWT (dev, n=3)	-0.0563	-0.0334	-0.0104
BWT (fresh, n=5)	-0.0362	-0.0313	-0.0038
BWT (combined, n=8)	-0.0438	-0.0320	-0.0063

Table 1: ACC and BWT means, reported at each validation stage rather than only the combined figure. aria-trl’s ACC is within noise of EWC at every stage; its BWT is the best of the three at every stage.

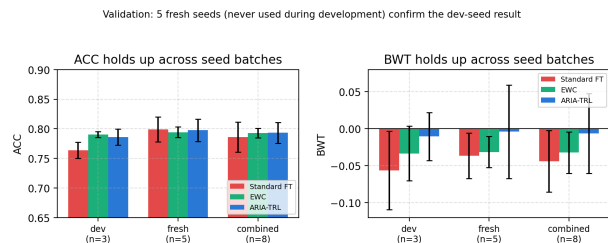


Figure 2: The same pattern — aria-trl tied with EWC on ACC, ahead on BWT — holds independently in the development seeds and the fresh, previously unseen seeds.

4.1 Where it is uneven

Backward Transfer’s standard deviation for aria-trl (0.0538 combined) is larger than EWC’s (0.0280) or Standard FT’s (0.0417) is not — this is not a clean sweep. Two seeds, 777 and 1234, are simultaneous weak points where aria-trl underperforms *both* baselines on *both* metrics. Figure 3 shows all 8 seeds individually rather than hiding this inside the aggregate: 5 of 8 seeds are clear aria-trl wins on retention, 3 of 8 are not.

5 Discussion

The accuracy race is a genuine tie, not a near-win rounded up. A 0.0005 combined-8 ACC delta is smaller than a single flipped prediction out of the 240 test examples evaluated per seed-batch stage. We are reporting this as “matches EWC,” not “beats EWC,” because the latter would overstate what eight seeds actually support.

The BWT advantage is real but not uniform. It holds in 5 of 8 seeds and holds in the combined aggregate at every validation stage (dev, fresh, combined) — but seeds 777 and 1234 show aria-trl losing on both metrics simultaneously to EWC. We did not tune against these seeds after seeing them, and we

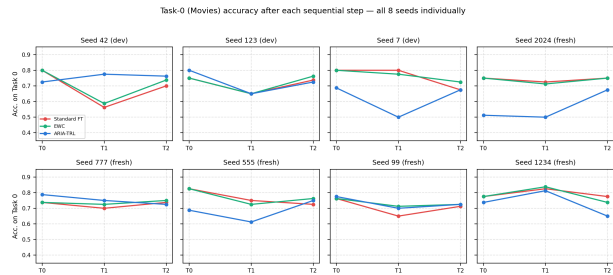


Figure 3: Task-0 accuracy after each step, all 8 seeds shown individually. Seeds 777 and 1234 are where aria-trl’s retention advantage does not hold.



Figure 4: Full accuracy matrix, seed 42. aria-trl’s off-diagonal (older-task) values sit closer to their diagonal values than either baseline’s.

are not tuning against them now; they are reported as the honest shape of the result, not smoothed over.

Why fix and re-validate, rather than just report the buggy version’s numbers? The pre-fix configuration, tested against EWC, showed aria-trl *losing* on accuracy to both baselines while winning only on BWT — a materially different and weaker claim than the one this report makes. We are documenting both, in this report, because a mechanism that only wins against a strawman baseline and a mechanism that ties a real one while forgetting less are different results, and conflating them would be the kind of thing that shouldn’t survive scrutiny.

6 Limitations

- **Scale.** 200 train / 80 test examples per task, CPU-only — chosen for iteration speed, not for representing production fine-tuning.
- **Task-identity assumption.** aria-trl’s per-task heads require knowing which task is active at evaluation time; EWC and Standard FT do not need this. Stated explicitly here (see Section 2).
- **Uneven BWT.** 3 of 8 seeds are simultaneous losses on both metrics; the aggregate advantage is real but not a guarantee per-seed.

- **Eight seeds.** More rigorous than three, still not enough for a strong statistical claim beyond "the direction is consistent across two independently-run seed batches."
- **Single model, single size, binary labels.** Unchanged from the first report: `distilgpt2` only, binary classification only.

Saravia, E., Liu, H.C.T., Huang, Y.H., Wu, J., and Chen, Y.S. CARER: Contextualized affect representations for emotion recognition. *EMNLP*, 2018.

7 Conclusion

Against a genuine EWC baseline, validated on 5 seeds held out from all development plus the original 3, `aria-trl` matches EWC’s accuracy within noise and cuts forgetting to near zero — a smaller, more precisely-stated claim than our first report’s, and a more trustworthy one, since it survived a fair comparison and an out-of-sample check rather than only an unprotected baseline. Code, the updated benchmark script, and full per-seed results for both validation stages are available at <https://github.com/rsd-darshan/ARIA-TRL>.

References

- Poudel, D. ARIA: Adaptive Recurrent Intelligence Architecture — Morphogenic Attention and Slow-Pathway Consolidation for Continual Learning. 2026. <https://github.com/rsd-darshan/ARIA>
- Poudel, D. `aria-trl`: Practical Continual Fine-Tuning for Language Models (first report). 2026. <https://github.com/rsd-darshan/ARIA-TRL>
- Kirkpatrick, J., Pascanu, R., Rabinowitz, N., et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.
- Buzzega, P., Boschini, M., Porrello, A., Abati, D., and Calderara, S. Dark experience for general continual learning: a strong, simple baseline. *Advances in Neural Information Processing Systems*, 33, 2020.
- Socher, R., Perelygin, A., Wu, J., et al. Recursive deep models for semantic compositionality over a sentiment treebank. *EMNLP*, 2013.
- Zhang, X., Zhao, J., and LeCun, Y. Character-level convolutional networks for text classification. *Advances in Neural Information Processing Systems*, 28, 2015.